

Практическая работа «Обнаружение объектов заданных классов на изображениях с использованием сверточных нейронных сетей»

Цель: изучить принципы и инструменты программной реализации обнаружения объектов заданных классов на изображениях с использованием существующих фреймворков, реализующих сверточные нейронные сети.

Содержание отчета: ответы на контрольные вопросы, выполненные задания и вывод о проделанной работе.

Теоретическая часть

Обнаружение объекта – это определение местоположения заданного объекта на изображении, при этом его размеры меньше размеров изображения, а количество объектов на изображении заведомо неизвестно.

Объекты на изображении задаются значениями некоторых признаков, которые представляются в виде вектора конечной размерности. Наборы признаков, т. е. длина векторов, являются одинаковыми для всех объектов. Совокупность признаков объекта определяется правилами его описания (например, описание полутонного изображения уровнями яркостей всех пикселей изображения или описание этого изображения его яркостной гистограммой). Соответственно, признаки могут выражаться в различными числовыми значениями.

На всём множестве объектов, которые могут присутствовать на изображениях, существует разбиение на подмножества, т. е. классы объектов. Объекты в пределах одного класса описываются наборами признаков, по которым они более схожи внутри этого класса и значительно отличаются от объектов других классов. Кроме этого, они имеют одно имя класса, по которому неразличимы. Например, класс «люди» и класс «автомобили».

Класс задается указанием некоторых признаков, присущих всем его членам. Соответственно, основными задачами являются выбор наиболее эффективного набора признаков для объектов и метода сравнения этих признаков для принятия решения об отношении объекта к определенному классу.

При обнаружении объектов на изображении, как правило, неизвестно количество искомым объектов, их расположение и размеры, что значительно усложняет задачу. Одним из первых методов обнаружения был подход на основе сопоставления с шаблоном, т. е. искомым объектов. Несмотря на то, что такой подход может обеспечить максимальную точность, он неустойчив даже к незначительным трансформациям искомого объекта на изображении

относительно исходного. Поэтому для описания объектов в дальнейшем использовали такие наборы признаков, как гистограммные признаки, ключевые точки объекта, признаки на основе спектральных преобразований (например, Фурье, Хаара, Адамара, вейвлет и др.). Такие подходы позволяют повысить возможность обнаружения отличных по масштабу и повороту объектов, однако все же не обеспечивают максимально возможную точность. Кроме этого, неясно как получить наиболее эффективный набор признаков.

Благодаря стремительному развитию вычислительной мощности компьютерной техники обнаружение объектов на изображениях все чаще выполняется с применением предварительно обученных сверточных нейронных сетей (СНС) на больших базах данных изображений, которые позволяют формировать эффективные наборы признаков объектов и их классифицировать. СНС в настоящее время обладают наилучшей обобщающей способностью.

СНС является одним из видов искусственных нейронных сетей (ИНС). ИНС – это существенно параллельно распределенный процессор, который обладает способностью к сохранению и репрезентации опытного знания. Она схожа с мозгом в двух аспектах: знание приобретается сетью в процессе обучения; для сохранения знания используются межнейронные соединения (синаптические соединения).

Работа ИНС состоит в преобразовании входных данных во времени, в результате чего меняется внутреннее состояние сети и формируются выходные воздействия. Обычно ИНС оперирует цифровыми, а не символьными величинами. Большинство моделей ИНС требуют обучения. В общем случае обучение – это такой выбор параметров сети, при котором сеть лучше всего справляется с поставленной проблемой.

Основным элементом нейронной сети является нейрон, который осуществляет операцию нелинейного преобразования суммы произведений входных сигналов на весовые коэффициенты:

$$y = \varphi(b_0 + \sum_{k,j} w_k x_j),$$

где $X = (x_1, x_2, \dots, x_n)$ – вектор входного сигнала;

$W = (w_1, w_2, \dots, w_n)$ – весовой вектор;

φ – функция активации;

b_0 – смещение.

Модель нейронного элемента изображена на рисунке 2.1. Каждому входу нейрона соответствует весовой коэффициент (синапс), который характеризует силу синаптической связи по аналогии с биологическим нейроном.

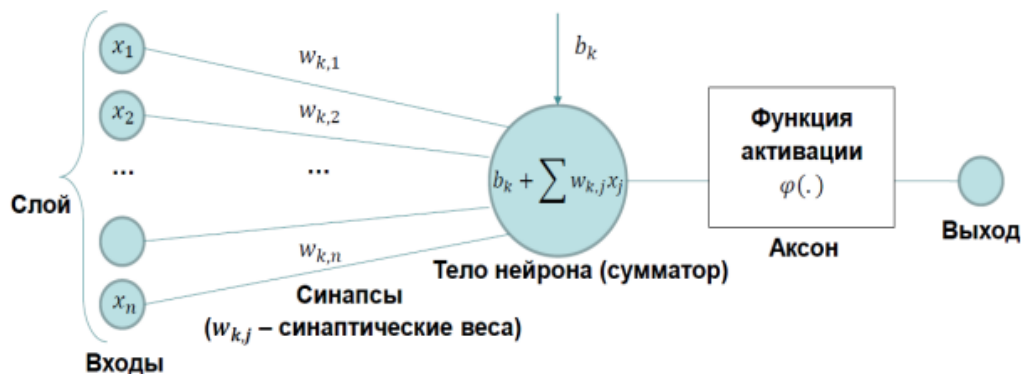


Рисунок 2.1. – Модель искусственного нейрона

Весовой вектор и функция активации определяют поведение любого нейрона, т. е. его реакцию на входные данные. Величина веса w_i определяет степень влияния входа i на выход нейрона, а знак – характер влияния. Положительные веса характерны для возбуждающих связей, способствующих повышению активности нейрона. Отрицательные, как правило, соответствуют тормозящим связям. Каждый из входов связан с некоторым источником информации (рецептор, формирующий признаки, распределительная ячейка, выход другого нейрона). Коэффициент b_0 является характеристикой, задающей начальный уровень активности (при нулевом входе). Изменение его эквивалентно смещению пороговой функции по оси абсцисс.

Функция активации используется для ограничения выхода нейрона в заданном диапазоне и для нелинейного преобразования взвешенной суммы. Последнее позволяет нейронному классификатору аппроксимировать любую нелинейную границу между классами в пространстве образов. Функция активации выбирается для конкретной задачи и является неизменной.

Веса конкретного нейрона являются настраиваемыми параметрами. Они содержат знания нейрона, определяющие его поведение. Процесс настройки этих знаний с целью получения нужного поведения называется обучением.

Слоем нейронной сети называется множество нейронных элементов, на которые в каждый такт времени параллельно поступает информация от других нейронных элементов сети. Помимо слоев нейронов часто используется понятие входного распределительного слоя. Распределительный слой передает входные сигналы на первый обрабатывающий слой нейронных элементов.

Сверточные нейронные сети (СНС) – это вид нейронных сетей, которые хотя бы в одном из своих слоев в качестве преобразования используют операцию свертки. Типичная архитектура СНС представляет собой многослойный каскад сетей прямого распространения, т. е. является однонаправленной, и включает сверточные слои, слои подвыборки (субдискретизации, пуллинга) и полносвязные слои. При этом используется преимущество двумерной структуры изображений и с помощью метода локальной связности, ограничивая количество связей между нейронами скрытого сверточного слоя и входными данными, т. е. каждый нейрон скрытого слоя связан только с локальным участком изображения. Сверточные слои основаны на математической операции свертки входного изображения с набором фильтров, которые описываются весами нейронов скрытого слоя. Данная операция в общем виде представляется как

$$y_{r,c,k} = \sum_{i=-H/2}^{H/2} \sum_{j=-W/2}^{W/2} \sum_{d=1}^D x_{i+r,j+c,d} \cdot w_{i,j,d,k} + b_k,$$

где (r, c) – положение фильтра на предыдущем слое;

x – выход нейрона предыдущего слоя;

H, W, D – высота, ширина и глубина k -го фильтра;

$w_{i,j,k,d}$ – веса фильтра;

b_k – вес связи с постоянным значением (смещение).

Таким образом, можно сказать, что *сверточный слой* – это набор карт (карты признаков – матрицы), сформированный на основе синаптического ядра (сканирующее ядро, фильтр), которое скользит по всей области предыдущей карты и находит определенные признаки объектов. *Шаг свертки* – это количество пикселей, на которые перемещается матрица фильтра по входному изображению. Если шаг равен единице, то ядро свертки смещается каждый раз на один пиксель, если шаг равен двум, то происходит смещение на два пикселя и т. п. Чем больше шаг, тем меньшего размера карты признаков получаются на выходе. Количество карт определяется требованиями к задаче: большое количество карт повышает качество распознавания, но увеличивает вычислительную сложность. Традиционно используется соотношение «один к двум», т. е. каждая карта предыдущего слоя связана с двумя картами сверточного слоя.

Размер у всех карт сверточного слоя одинаковый и вычисляется как

$$(w, h) = (mW - kW + 1, mH - kH + 1),$$

где mW, mH – ширина и высота предыдущей карты соответственно;
 kW, kH – ширина и высота ядра соответственно.

Каждое ядро находит определенные признаки объектов, например, это могут быть вертикальные или горизонтальные линии, или же линии, расположенные под определёнными углами. Ядро представляет собой систему разделяемых весов или синапсов и является одной из главных особенностей СНС. В обычной многослойной ИНС много связей между нейронами, что замедляет процесс обнаружения. В СНС общие веса позволяют сократить число связей и находить один и тот же признак по всей области изображения.

Размер ядра нечетный от 3×3 до 7×7 , так как обработка ведется относительно центрального элемента ядра. Необходимо чтобы размер карт сверточного слоя был четным для исключения потери информации при уменьшении размерности в подвыборочном слое. Малый размер ядра не сможет выделить какие-либо признаки большего размера, а при большом размере ядра увеличивается количество связей между нейронами и, соответственно, время обучения и работы СНС.

Подвыборочный слой также имеет карты, их количество совпадает с количеством карт предыдущего сверточного слоя. Подвыборочный слой служит для уменьшения размерности карт предыдущего слоя. Такой подход позволяет снизить вариативность данных, повысить устойчивость СНС к небольшим изменениям изображений в пределах локальной области.

В процессе сканирования карты предыдущего сверточного слоя ядром подвыборочного слоя сканирующее ядро на нем не пересекается, что является отличием от организации свертки. Обычно каждая карта имеет ядро размером 2×2 , и это позволяет уменьшить предыдущие карты сверточного слоя в два раза.

Для этого вся карта признаков разделяется на ячейки 2×2 элемента, из которых выбираются определенные значения. Наиболее используемые два типа объединения: максимальное (max pooling) и среднее (average pooling) (рисунок 2.2). Первое возвращает максимальное значение из покрытой ядром части изображения, второе возвращает среднее значение из всех значений покрытой ядром части.

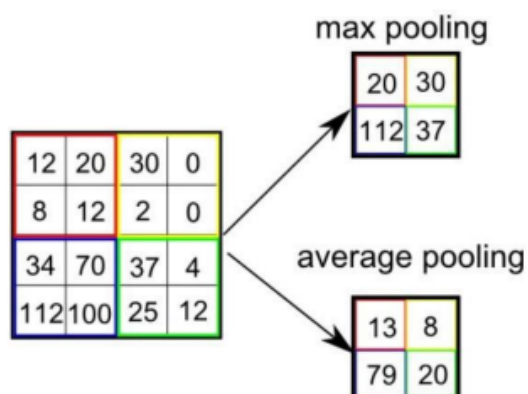


Рисунок 2.2. – Примеры типов объединений слоя подвыборки

Результат свертки после слоя подвыборки поступает на функцию активации, которая определяет выходной сигнал и является нелинейной, т. е. позволяет оставлять для дальнейшего анализа только значимые нейроны, в качестве которых желательно использовать нелинейные активационные функции, например, сигмоидального вида. Следует отметить, что широко используются активационные функции, состоящие из линейных участков, например, RELU (rectified linear unit) и PReLU, которая использует наклон отрицательной части в качестве параметра в процессе обучения с использованием алгоритма обратного распространения ошибки, что позволяет увеличить точность работы СНС.

Формально данный этап может быть описан выражением

$$y_p^l = \varphi \left(a^l \cdot \beta \left(x^{l-1} \right) + b^l \right),$$

где y_p^l – выход текущего подвыборочного слоя;

φ – функция активации;

a^l, b^l – коэффициенты сдвига;

β – функция выборки значений из локальной области.

Слои свёртки и подвыборки вместе образуют i -й слой свёрточной нейронной сети. За счет использования нескольких попеременных слоев свертки и подвыборки СНС позволяет получать представления, независимые от конкретного расположения локального признака в изображении, и одинаковым образом реагировать на интересующие объекты, т. е. обнаруживать объекты на любом участке изображения. Количество таких слоёв может варьировать в зависимости от сложности изображений, чтобы точнее выделять признаки, однако увеличение слоев требует увеличения вычислительной мощности и большего объема памяти.

СНС обеспечивает инвариантность по отношению к сдвигу, однако не предусматривает устойчивости к другим аффинным преобразованиям, например, вращению или зеркальному отражению. Для решения этой проблемы, как правило, используются эвристические методы, такие как выравнивание изображения по линии горизонта, использование многомасштабного представления и различных отражённых копий. Последний блок СНС является многослойным персептроном, машиной опорных векторов или другим классификатором. Он служит для задачи классификации, моделирует сложную нелинейную функцию, оптимизацией которой улучшается качество распознавания. Нейроны каждой карты предыдущего подвыборочного слоя связаны с одним нейроном скрытого слоя. Связи могут быть разными, например: только часть нейронов какой-либо из карт подвыборочного слоя быть связана с первым нейроном скрытого слоя, а оставшаяся часть со вторым, либо все нейроны первой карты связаны с нейронами первого и второго скрытого слоя. Пример топологии СНС для многоклассовой классификации показан на рисунке 2.3.

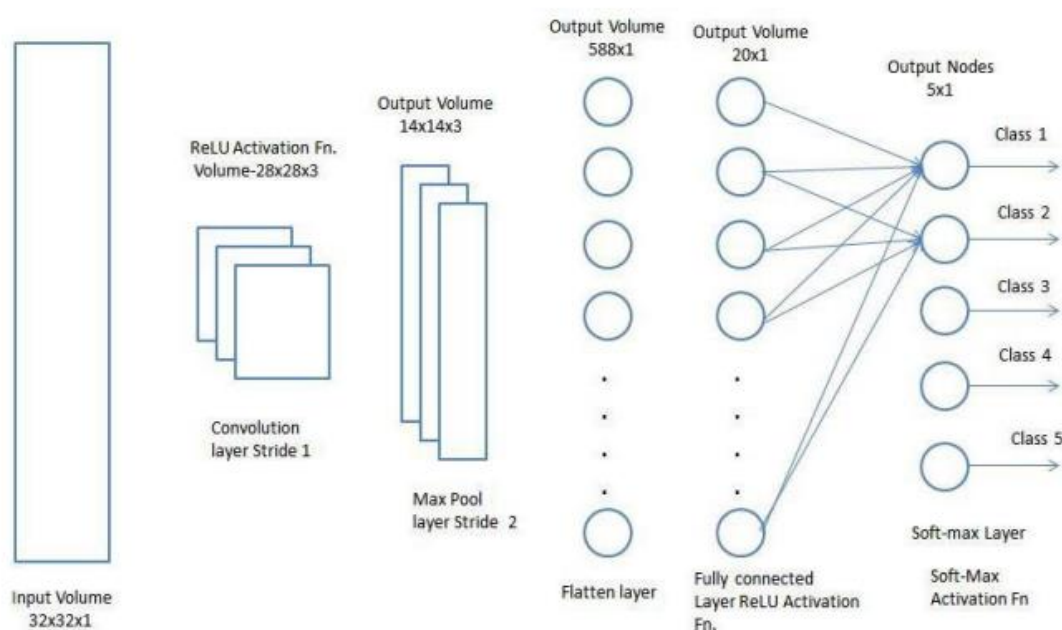


Рисунок 2.3. – Топология СНС для многоклассовой классификации

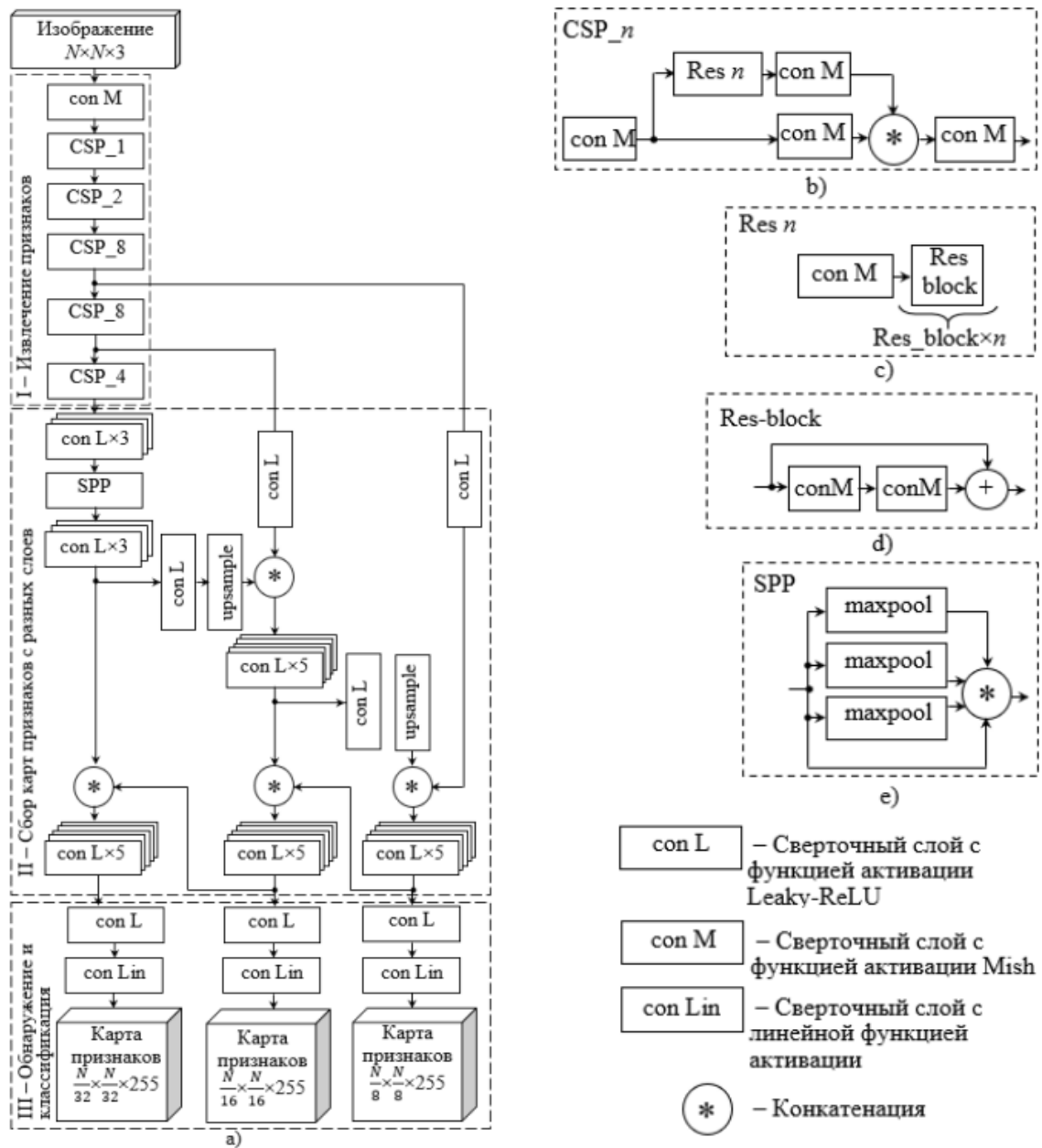
Среди известных моделей СНС, отличающихся архитектурой, точностью и быстродействием, для обнаружения объектов на изображениях и видеопоследовательностях следует выделить двухэтапные СНС, которые предполагают на первом этапе локализацию регионов-претендентов, а на втором этапе выполняется их классификация. Однопроходные СНС позволяют осуществлять локализацию и классификацию в составе одной модели, и не требуют обработки одного и того же изображения повторно.

Среди двухпроходных следует выделить модификации R-CNN (Region-based Convolutional Neural Networks) и ResNet (Residual Networks). Первая версия модели R-CNN основана на предварительном выделении регионов на изображении, вычислении признаков с использованием СНС (например, AlexNet) и применении классификатора для идентификации объектов. Данная модель получила развитие в версиях Fast R-CNN и Faster R-CNN. Архитектура Fast R-CNN направлена на уменьшение использования числа регионов на изображении за счет применения слоя ROI, который действует по принципу слоев субдискретизации, позволяя получать области уменьшенного размера. В Faster R-CNN предполагается выполнение двух этапов. На первом из них определяются области на изображении, в которых предположительно могут быть расположены объекты, с применением глубокой полносвязной сети (Region Proposal Network). На втором этапе используется детектор Fast R-CNN, который выполняет поиск объектов в предложенных регионах. Модель Faster R-CNN неустойчива к зашумленным изображениям и требует значительных вычислительных затрат.

Архитектура СНС ResNet использует пропускающие (residual) соединения, которые позволяют минимизировать ухудшение качества работы при увеличении количества слоев СНС, если на некотором слое сети достигнут предел точности. Коэффициент ошибок составляет 3,57% в метрике top 5. В работе представлена модель Inception-ResNet как развитие модели Inception путем ввода замыкающих соединений (shortcut connection), которые при необходимости позволяют пропускать слой и, соответственно, обнулять его влияние на результат работы детектора. Это даёт возможность изменять архитектуру сети так, чтобы конечное количество слоёв определялось для конкретной задачи в процессе обучения, что позволяет уменьшить коэффициент ошибки до 3,1% в метрике top 5. Вместе с тем представленные модели характеризуются высокими временными затратами. Поэтому для решения многих задач используются усечённые варианты архитектур с уменьшенным количеством слоев (например, ResNet-34, ResNet-50 и др.), которые, однако, не позволяют достичь точности базовой модели.

Семейство СНС YOLO относится к однопроходным и даёт возможность осуществлять локализацию и идентификацию объектов одной моделью. Алгоритм обнаружения объектов YOLOv3 обладает достаточно высокой точностью, однако скорость работы меньше по сравнению с предыдущими версиями YOLO, и разработка YOLOv4 [6] была направлена на увеличение

скорости работы и оптимизацию параллельных вычислений. Структура¹ YOLOv4 представлена на рисунке 2.4.



а) – общая схема; б) – структура блока с CSP-соединением;
 в) – объединение n Res-блоков; д) – структура Res-блока; е) – блок SPP

Рисунок 2.4. – Архитектура YOLOv4

Структуру YOLOv4 можно разделить на три основных блока: блок извлечения признаков, который служит для выявления характерных особенностей объектов на входном изображении; блок сбора карт признаков с разных

слоев, который собирает и передает карты признаков с различных уровней нейронной сети на блок обнаружения и классификации, который, в свою очередь, формирует выходные карты признаков для разных масштабов, что позволяет предсказывать координаты ограничительных рамок и классифицировать содержимое каждой ячейки на входном изображении.

На рисунке 2.4, а) блок извлечения признаков представляет собой CSPDarknet-53, в основе которой находится Darknet-53, состоящая из 53 сверточных слоев. Отличие заключается в использовании межэтапных соединений (CSP – Cross-Stage-Partial connection) для снижения вычислительной сложности. На рисунке 2.4, б) представлена структура блока с CSP-соединением. В данном блоке карты признаков разделяются на две части: одна из них проходит через группу Res-блоков (Residual blocks), другая поступает на сверточный слой, после чего выходные данные объединяются. Res-блоки представлены в группах по 1, 2, 4 или 8 блоков (см. рисунок 2.4, с), каждый из которых состоит из сверточных слоев 1×1 и 3×3 с замыкающим соединением (shortcut connection) (см. рисунок 2.4, d). Использование такого соединения обеспечивает альтернативный путь для градиента, что приводит к лучшей сходимости модели. Другим отличием CSPDarknet-53 является использование функции активации Mish, которая представляет собой комбинацию из функции идентичности, гиперболического тангенса, softplus и характеризуется достаточно низкой вычислительной сложностью и определяется как

$$f(x) = x \tanh(\text{softplus}(x)) = x \tanh(\ln(1 + e^x)).$$

Кроме этого, Mish является немонотонной, гладкой и непрерывной функцией, неограниченной сверху, но ограниченной снизу. Отсутствие верхней границы позволяет избегать насыщения, которое может приводить к замедлению обучения, а значит позволяет ускорить процесс обучения. Наличие нижней границы обеспечивает эффект регуляризации. Немонотонность сохраняет небольшие отрицательные значения, что стабилизирует градиентный поток, а гладкость и непрерывность эффективны при обобщении и оптимизации результатов.

Блок сбора карт признаков с разных слоев включает модифицированный модуль SPP (SPP – Spatial Pyramid Pooling) (см. рисунок 2.4, е) для увеличения рецептивного поля и модифицированный модуль PAN для создания пирамиды признаков. Первый из них выполняет операцию maxpool по картам

признаков $\frac{N}{32} \times \frac{N}{32} \times 255$ с разным размером ядра $k = \{1, 5, 9, 13\}$, но идентичным заполнением, которое используется для сохранения пространственного размера. Затем четыре комплекта соответствующих карт признаков объединяются в один, размером $\frac{N}{32} \times \frac{N}{32} \times 2048$. Это увеличивает область анализа, улучшая таким образом точность модели, при этом без существенных вычислительных затрат.

Для агрегирования характеристик объектов в YOLOv4 используется модифицированная версия PAN (Path Aggregation Networks), использующая пирамиды признаков, которые служат для объединения карт признаков нижних и верхних уровней сети, то есть сочетают семантическую информацию с верхних уровней и более точную с нижних. В этой модификации PAN вместо операции суммирования результатов соседних слоев используется конкатенация, что позволяет увеличить точность прогнозов.

Блок обнаружения и классификации используется для нахождения ограничительной рамки и классификации содержимого в каждой ячейке. Изображение размером $N \times N \times 3$, подаваемое на вход нейронной сети, разделяется на $S \times S$ ячеек – которые представляют собой область интереса (RoI – Region of Interest). Количество ячеек прямо пропорционально размеру входного изображения и обратно пропорционально шагу дискретизации на каждом из масштабов. Карты признаков на выходе сети YOLOv4 используют шаги дискретизации 32, 16 и 8, которые показывают во сколько раз уменьшился размер выходного слоя относительно входного изображения и позволяют осуществлять обнаружение в трех масштабах. Карты признаков на каждом из трех масштабов будут иметь размер $S \times S \times (N_B \times (5 + C))$, где N_B – количество ограничительных рамок, которые могут быть предсказаны для каждой ячейки, C – количество классов объектов. В YOLOv4 для каждой ячейки используется 3 anchor-рамки, представляющие собой прямоугольники различных размеров с разным соотношением сторон. Вектор признаков каждой ячейки предсказывает 85 параметров для трех anchor-рамок: предсказывается смещение координат t_x и t_y и отклонения размеров t_w и t_h , на основании которых определяется положение предсказанных рамок (рисунок 2.5), вероятность обнаружения объекта и вероятность принадлежности объекта к классу.

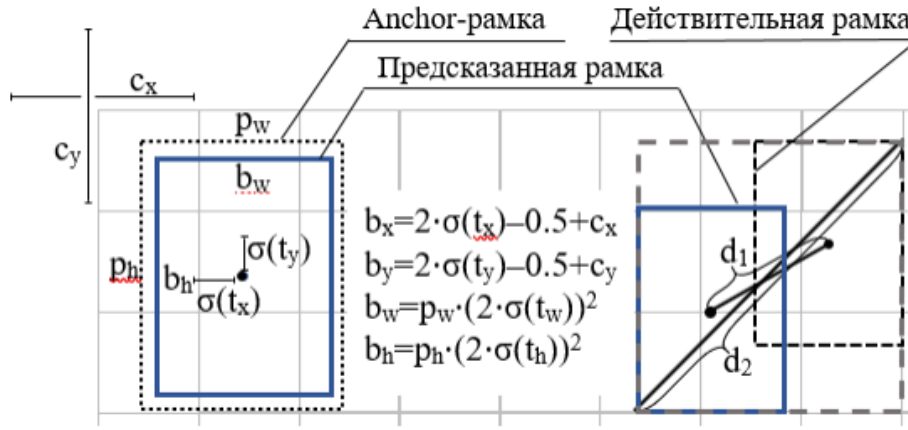


Рисунок 2.5. – Построение ограничительных рамок при обнаружении объектов:

c_x, c_y — координаты верхней левой ячейки; p_h и p_w — размеры anchor-рамки; b_x, b_y, b_w, b_h — координаты центра и размеры предсказанной ограничительной рамки; $\sigma(t)$ — функция сигмоиды, t_x, t_y, t_w, t_h — отклонение координат и размеров предсказанной рамки; d_1 — расстояние между центральными точками действительной и предсказанной рамки, d_2 — диагональ минимальной рамки, охватывающей две рамки

Тогда вектор признаков для каждой ячейки изображения с учетом трех anchor-рамок будет равен $1 \times 1 \times 255$. На основании вектора признаков определяются параметры предсказанной рамки. Предсказанная рамка также имеет 85 параметров: P_c — вероятность правильного нахождения ограничительной рамки, b_x и b_y — координаты центра, b_h и b_w — высота и ширина предсказанной ограничительной рамки, $c_1 \dots c_{80}$ — вероятность нахождения объекта одного из 80 классов. Класс объекта, ограниченного предсказанной рамкой, определяется с использованием $c_1 \dots c_{80}$ и сигмоидальной функции. Для определения положения ограничительных рамок применяется функция потерь L_{Clou} :

$$L_{Clou} = 1 - IoU + \frac{d_1^2}{d_2^2} + \alpha v,$$

где b и b_{gt} — центральные точки B и B_{gt} . $B = (b_x, b_y, b_w, b_h)$ — предсказанная рамка объекта, $B_{gt} = (x_{gt}, y_{gt}, w_{gt}, h_{gt})$ — действительная рамка;

d_1 — евклидово расстояние между центральными точками предполагаемой и действительной ограничительной рамки;

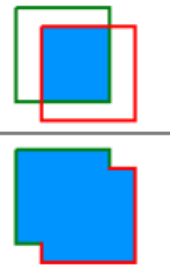
d_2 — диагональ минимальной рамки, которая формируется для предсказанной и действительной ограничительных рамок;

α – параметр, зависящий от взаимного расположения предсказанной и действительной рамки:

$$\alpha = \begin{cases} 0, & IoU < 0,5; \\ \frac{v}{(1 - IoU) + v}, & IoU \geq 0,5, \end{cases}$$

где $v = \frac{4}{\pi^2} \left(\arctan \frac{w_{gt}}{h_{gt}} - \arctan \frac{b_w}{b_h} \right)^2$ – мера согласованности сторон ограничительных рамок;

IoU – отношение между пересечением и объединением предсказанной и действительной рамок, т. е. для прямоугольников, описывающих найденные (R) и аннотированные (O) объекты на изображениях базы данных:

$$IoU(O, R) = \frac{O \cap R}{O \cup R} = \frac{\text{area of intersection}}{\text{area of union}}$$


Объект считается правильно обнаруженным, если $(IoU(O, R)) \geq 0,5$. Примеры перекрытия объектов для $(IoU(O, R))$ показаны на рисунке 2.6.

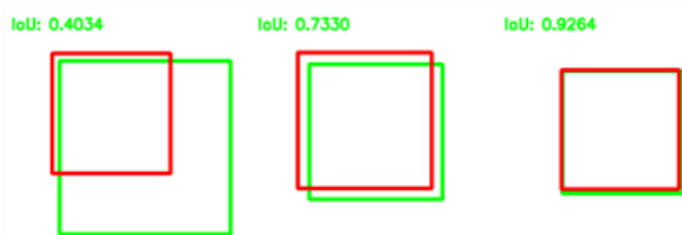


Рисунок 2.6. – Примеры перекрытия объектов и значения метрики (IoU)

При расчете потерь учитывается площадь перекрытия, расстояние между центральными точками и соотношение сторон. L_{CioU} позволяет увеличить площадь перекрытия действительной и предсказанной ограничительной рамки, минимизировать расстояние между центральными точками и сохранить постоянное соотношение сторон. L_{CioU} обеспечивает лучшую скорость и точность в задаче приближения предсказанной ограничительной рамки к истинному положению по сравнению с L_{DioU} (Distance-IoU loss) и L_{GioU} (Generalized-IoU loss) потерями.

Подавление немаксимумов (NMS) применяется для обработки ситуации, когда для одного и того же объекта предсказано несколько рамок с высокими вероятностями соответствия. В YOLOv4 используется подход на основе greedy NMS, которая следует гипотезе о том, что обнаруженные предполагаемые рамки с большим количеством перекрытий соответствуют одному и тому же объекту. При этом рамки-кандидаты сортируются по степени достоверности классификации и удаляются дублирующие.

При обнаружении объектов могут быть следующие ситуации:

- искомые объекты обнаружены правильно, т. е. истинно-положительные случаи или правильные обнаружения (True Positive, TP);
- ложные объекты обнаружены как искомые, т. е. ложно-положительные случаи или ложные обнаружения (False Positive, FP);
- ложные объекты обнаружены как ложные, т. е. истинно-отрицательные случаи (True Negative, TN);
- искомые объекты обнаружены как ложные, т. е. ложно-отрицательные случаи или пропуски (False Negative, FN).

Для оценки эффективности обнаружения объектов используют следующие показатели (метрики):

- точность (precision) определяет долю правильно обнаруженных объектов к общему числу срабатываний детектора:

$$p = \frac{TP}{TP + FP};$$

- полнота (recall) характеризует долю объектов, реально относящихся к искомому классу, обнаруженных верно:

$$r = \frac{TP}{TP + FN}.$$

На основе полученных значений рассчитывается средняя точность (AP) обнаружения объектов для каждого класса для 11 пороговых уровней в диапазоне от 0 до 1 с шагом 0,1 по формуле:

$$AP = \frac{1}{11} \cdot \sum_{i=0}^{10} p_{\text{int}}(r_i),$$

где $p_{\text{int}}(r_i) = \max p(\tilde{r}), \tilde{r} \geq r_i$.

При вычислении метрики mAP (meanAveragePrecision), которая предполагает усреднение значений метрики AP (AveragePrecision) по классам, значения полученной средней точности усредняются для заданных классов объектов.

Наиболее широко в качестве баз данных для тестирования алгоритмов обнаружения объектов используются:

- PASCAL VOC 2007, 2012 – изображения 20 классов объектов [7];
- MS COCO – набор изображений, содержащих разметку для 80 категорий объектов [8];
- Open Images Dataset – изображения, содержащие разметку 600 классов объектов [9];
- Street View House Numbers (SVHN) – изображения с разметкой окаймляющих прямоугольников для цифр, содержащихся на естественных надписях (в частности, на номерах домов) [10];
- Stanford Dogs Dataset – изображения собак 120 пород [11].

Практическая часть

Задание 1. Реализовать обнаружение объектов на изображениях с использованием СНС YOLOv4. Для этого использовать файлы:

- yolov4.cfg – файл конфигурации, который описывает архитектуру используемой сети;
- yolov4.weights – файл весовых коэффициентов для обученной модели;
- coco.names – файл, содержащий имена классов объектов для изображений из базы данных MS COCO, на которых была обучена модель.

** Имена классов представлены в приложении 1.*

Указанные файлы расположены в репозитории Darknet на GitHub: <https://github.com/AlexeyAB/darknet>

Тестовый набор MS COCO доступен по ссылке: <http://images.cocodataset.org/zips/test2017.zip>

В приложении 2 представлены фрагменты кода на языке Python, для реализации алгоритма обнаружения объектов заданных классов.

Размер входного слоя должен быть кратен 32, т. к. в YOLOv4 по мере прохождения по сети входное изображение со сторонами $N \times N$ уменьшается с шагом дискретизации d , равным 8, 16 и 32, и на каждом из трех масштабов предсказания входной ячейке размером N/d будет соответствовать вектор признаков с размерностью $1 \times 1 \times K$, где K – количество карт признаков размером $N/d \times N/d$.

Степень уверенности сети (confidence) отражает вероятность того, что ограничительная рамка объекта предсказана верно, и пороговое значение степени уверенности (confidenceThreshold) определяет будет ли эта ограничительная рамка представлена как результат обнаружения.

Задание 2. Вычислить точность и полноту детектора при обнаружении объектов на 10 изображениях из базы данных MS COCO для различных степеней уверенности детектора в заданном диапазоне и с определенным шагом, например, от 0,3 до 0,9 с шагом 0,2. Каждое изображение должно содержать не менее 3 объектов разного класса. Данные представить в виде таблицы.

Таблица. – Представление результатов обнаружения объектов

Уверенность СНС	1		2		...	10	
	<i>p</i>	<i>r</i>	<i>p</i>	<i>r</i>		<i>p</i>	<i>r</i>

Пример обнаружения объектов на изображении показан на рисунке 1. Согласно визуальной оценке на изображении, представленном на этом рисунке, верно детектировано девять объектов, ложного обнаружения не было, пропущен один объект, т. к. не обнаружен один из четырех автомобилей в мозаике.

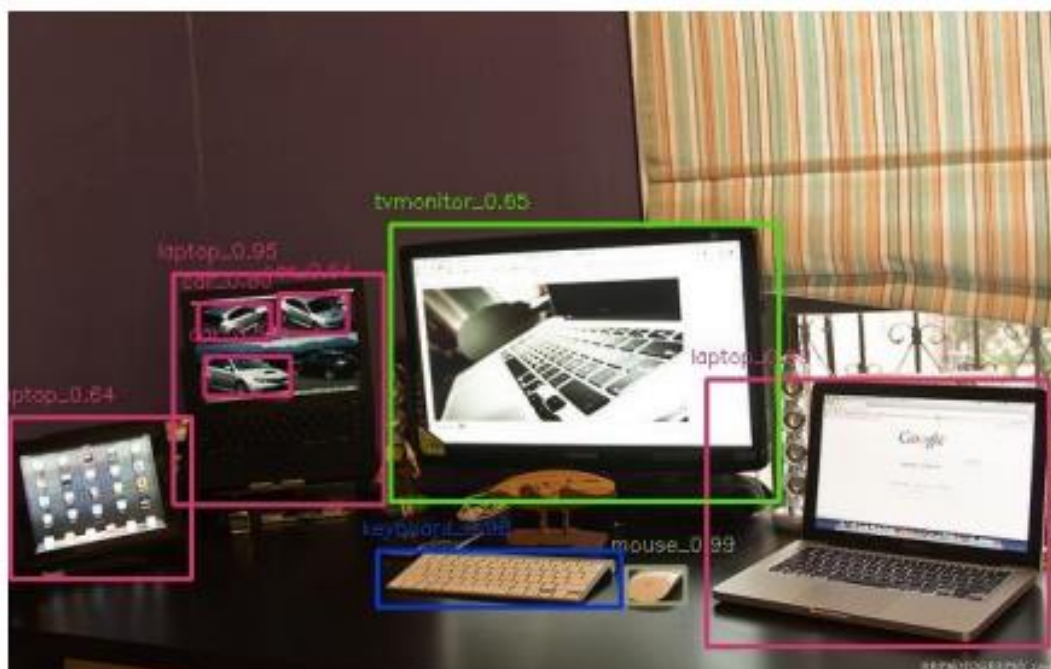


Рисунок 1. – Пример обнаружения объектов различных классов на изображении

Задание 3. Внести изменения в код для обнаружения объектов только заданных классов по вариантам. Для обнаружения объектов только заданных классов необходимо добавить условие проверки соответствия требуемому номеру класса. Например, если требуется обнаружение объектов, которые принадлежат только классам «автомобиль» и «человек», то следует добавить строку:

```
if class_id == 0 or class_id == 2:
```

после определения класса обнаруженного объекта в строке 46 с соблюдением табуляции. Следует учитывать, что нумерация классов

начинается с 0.

Для детектирования использовать параметры СНС, определенные как наилучшие в пункте 2. Представить примеры обнаружения заданных объектов на изображениях, на которых присутствуют объекты пяти разных классов или более.

№ Варианта	Описание
Вариант 1	Велосипед и мотоцикл
Вариант 2	Автомобиль и грузовик
Вариант 3	Поезд и автобус
Вариант 4	Кот и собака
Вариант 5	Лошадь и корова
Вариант 6	Рюкзак и зонтик
Вариант 7	Бейсбольная бита и бейсбольная перчатка
Вариант 8	Птица и воздушный змей
Вариант 9	Яблоко и банан
Вариант 10	Зубная щетка и фен для волос
Вариант 11	Микроволновая печь и духовка
Вариант 12	Светофор и пожарный гидрант
Вариант 13	Человек и сумка
Вариант 14	Собака и овцы
Вариант 15	Скамейка и знак остановки

Перечень контрольных вопросов

- 1- Назначение и примеры функций активации
- 2- Состав модели искусственного нейрона.
- 3- Приведите пример вычисления свертки для фрагмента изображения и произвольного ядра слоя.
- 4- Что понимается под задачей обнаружения объектов на изображении?
- 5- В чем преимущества моделей семейства Yolo?

Приложение 1

Классы объектов изображений в базе данных MS COCO

1	person	человек	41	wine glass	бокал
2	bicycle	велосипед	42	cup	чашка
3	car	автомобиль	43	fork	вилка
4	motorbike	мотоцикл	44	knife	нож
5	aeroplane	самолет	45	spoon	ложка
6	bus	автобус	46	bowl	миска
7	train	поезд	47	banana	банан
8	truck	грузовик	48	apple	яблоко
9	boat	лодка	49	sandwich	бутерброд
10	traffic light	светофор	50	orange	апельсин
11	fire hydrant	пожарный гидрант	51	broccoli	брокколи
12	stop sign	знак остановки	52	carrot	морковь
13	parking meter	парковочный счетчик	53	hot dog	хот-дог
14	bench	скамейка	54	pizza	пицца
15	bird	птица	55	donut	пончик
16	cat	кот	56	cake	торт, пирог
17	dog	собака	57	chair	стул
18	horse	лошадь	58	sofa	диван
19	sheep	овца	59	potted plant	растение в горшке
20	cow	корова	60	bed	кровать
21	elephant	слон	61	dining table	обеденный стол
22	bear	медведь	62	toilet	туалет
23	zebra	зебра	63	tvmonitor	монитор
24	giraffe	жираф	64	laptop	ноутбук
25	backpack	рюкзак	65	mouse	мышь
26	umbrella	зонт	66	remote	пульт
27	handbag	сумка	67	keyboard	клавиатура
28	tie	галстук	68	cell phone	мобильный телефон
29	suitcase	чемодан	69	microwave	микроволновая печь
30	frisbee	фрисби	70	oven	духовка
31	skis	лыжи	71	toaster	тостер
32	snowboard	сноуборд	72	sink	раковина
33	sports ball	спортивный мяч	73	refrigerator	холодильник
34	kite	воздушный змей	74	book	книга
35	baseball bat	бейсбольная бита	75	clock	часы
36	baseball glove	бейсбольная перчатка	76	vase	ваза
37	skateboard	скейтборд	77	scissors	ножницы
38	surfboard	доска для серфинга	78	teddy bear	плюшевый мишка
39	tennis racket	теннисная ракетка	79	hair drier	фен для волос
40	bottle	бутылка	80	toothbrush	зубная щетка

Особенности и примеры программной реализации поиска объектов

Подключение модуля OpenCV dnn для работы с фреймворком Darknet:

```
net = cv2.dnn.readNetFromDarknet(config_file, weights_file)
```

Чтение классов объектов из файла:

```
with open(classes_file, 'r') as f:
    classes = [line.strip() for line in f.readlines()]
```

Для каждого значения степени уверенности сети, для каждого из выбранных изображений создается blob-объект, который затем передается в сеть для обработки:

```
confidenceThreshold = [0.3, 0.5, 0.7, 0.9]
for th in confidenceThreshold:
    for image_file in images:
        image = cv2.imread(image_file)
        width = image.shape[1]
        height = image.shape[0]
        blob = cv2.dnn.blobFromImage(image, 0.00319, (s[0],
s[1]), (0, 0, 0), True, crop=False)
# передача изображения в сеть
        net.setInput(blob)
        ln = net.getLayerNames()
```

YOLOv4 имеет 3 несвязанных выхода, и для получения предсказаний на этих выходах, обнаружения регионов-кандидатов для объектов и получения координат ограничительных рамок для лучших предсказаний используется алгоритм подавления немаксимумов NMS:

```
ln = [ln[i[0] - 1] for i in net.getUnconnectedOutLayers()]
outs = net.forward(ln)
class_ids = []
confidences = []
boxes = []
nms_threshold = 0.4
# Для каждого из 3-х выходных слоев YOLOv4:
for out in outs:
    for detection in out:
        scores = detection[5:]
```



```

class_id = np.argmax(scores)
confidence = scores[class_id]
if confidence > th:
    center_x = int(detection[0] * width)
    center_y = int(detection[1] * height)
    w = int(detection[2] * width)
    h = int(detection[3] * height)
    x = center_x - w / 2
    y = center_y - h / 2
    class_ids.append(class_id)
    confidences.append(float(confidence))
    boxes.append([x, y, w, h])

# Определение координат предсказанных объектов:
indices = cv2.dnn.NMSBoxes(boxes, confidences, th,
nms_threshold)
for i in indices:
    i = i[0]
    box = boxes[i]
    x = box[0]
    y = box[1]
    w = box[2]
    h = box[3]

```

Нанесение на изображение ограничительной рамки обнаруженных объектов с указанием предсказанного класса объекта и степени уверенности сети в своем предсказании:

```

draw_bounding_box(image, class_ids[i], confidences[i],
round(x), round(y), round(x + w), round(y + h))

```